

viz-1

October 6, 2025

0.0.1 Data Visualization

```
[3]: import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns
```

```
[62]: # Data: https://d2beiqkhq929f0.cloudfront.net/public_assets/assets/000/021/299/
↳ original/final_vg1_-_final_vg_%281%29.csv?1670840166
```

```
[4]: data = pd.read_csv('final_vg.csv')
```

```
[5]: data.head()
```

```
[5]:
```

	Rank	Name	Platform	Year	Genre	\
0	2061	1942	NES	1985.0	Shooter	
1	9137	¡Shin Chan Flipa en colores!	DS	2007.0	Platform	
2	14279	.hack: Sekai no Mukou ni + Versus	PS3	2012.0	Action	
3	8359	.hack//G.U. Vol.1//Rebirth	PS2	2006.0	Role-Playing	
4	7109	.hack//G.U. Vol.2//Reminisce	PS2	2006.0	Role-Playing	

	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
0	Capcom	4.569217	3.033887	3.439352	1.991671	12.802935
1	505 Games	2.076955	1.493442	3.033887	0.394830	7.034163
2	Namco Bandai Games	1.145709	1.762339	1.493442	0.408693	4.982552
3	Namco Bandai Games	2.031986	1.389856	3.228043	0.394830	7.226880
4	Namco Bandai Games	2.792725	2.592054	1.440483	1.493442	8.363113

```
[6]: data.shape
```

```
[6]: (16652, 11)
```

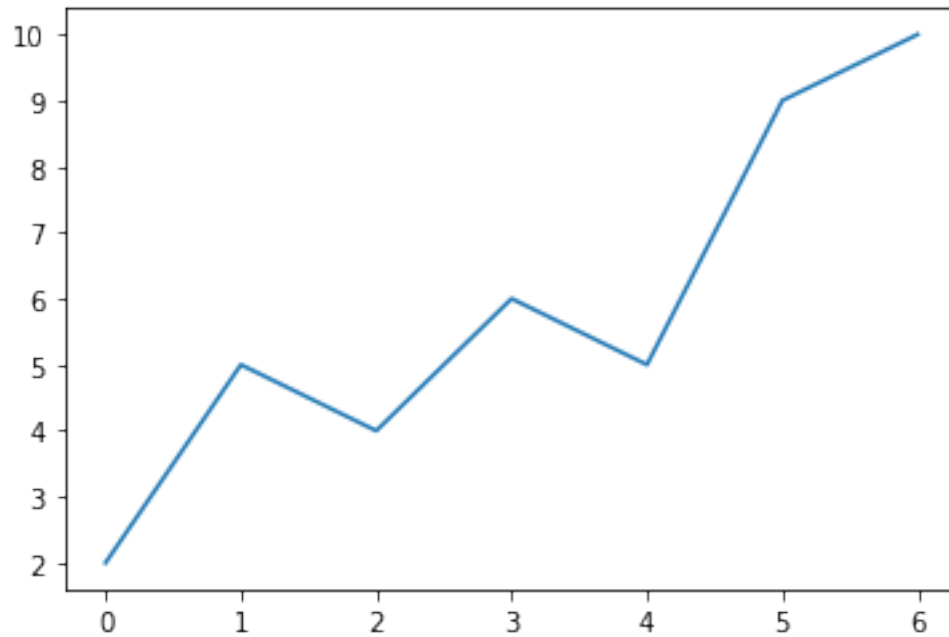
```
[ ]:
```

```
[ ]:
```

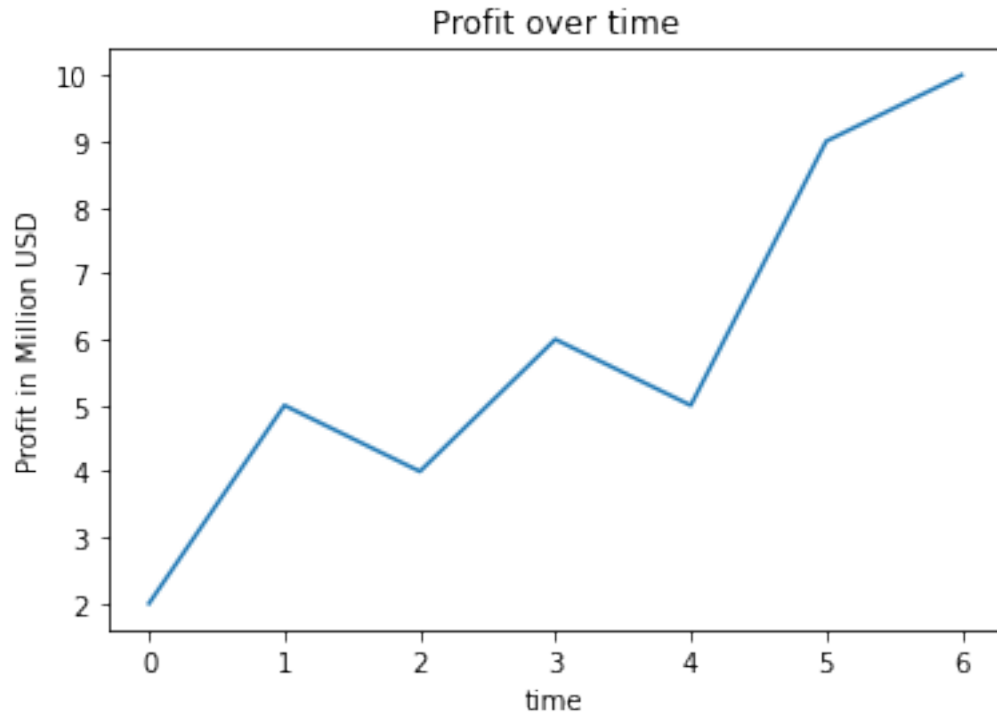
0.0.2 Line chart

```
[8]: x_val = [0, 1, 2, 3, 4, 5, 6] # time  
     y_val = [2, 5, 4, 6, 5, 9, 10] # stock price
```

```
[10]: plt.plot(x_val, y_val)  
      plt.show()
```



```
[12]: plt.plot(x_val, y_val)  
  
      plt.title("Profit over time")  
      plt.xlabel("time")  
      plt.ylabel("Profit in Million USD")  
  
      plt.show()
```



```
[ ]:
```

0.0.3 Categorical

```
[13]: data.head()
```

```
[13]:
```

	Rank	Name	Platform	Year	Genre	\
0	2061	1942	NES	1985.0	Shooter	
1	9137	¡Shin Chan Flipa en colores!	DS	2007.0	Platform	
2	14279	.hack: Sekai no Mukou ni + Versus	PS3	2012.0	Action	
3	8359	.hack//G.U. Vol.1//Rebirth	PS2	2006.0	Role-Playing	
4	7109	.hack//G.U. Vol.2//Reminisce	PS2	2006.0	Role-Playing	

	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
0	Capcom	4.569217	3.033887	3.439352	1.991671	12.802935
1	505 Games	2.076955	1.493442	3.033887	0.394830	7.034163
2	Namco Bandai Games	1.145709	1.762339	1.493442	0.408693	4.982552
3	Namco Bandai Games	2.031986	1.389856	3.228043	0.394830	7.226880
4	Namco Bandai Games	2.792725	2.592054	1.440483	1.493442	8.363113

```
[16]: data['Genre'].unique()
```

```
[16]: array(['Shooter', 'Platform', 'Action', 'Role-Playing', 'Racing', 'Misc',  
        'Adventure', 'Sports', 'Puzzle', 'Simulation', 'Strategy',  
        'Fighting'], dtype=object)
```

```
[58]: data['Genre'].value_counts()
```

```
[58]: Action          3316  
Sports            2400  
Misc              1739  
Role-Playing     1488  
Shooter          1310  
Adventure        1286  
Racing           1249  
Platform          886  
Simulation        867  
Fighting          848  
Strategy          681  
Puzzle            582  
Name: Genre, dtype: int64
```

```
[59]: data['Genre'].value_counts(ascending=True)
```

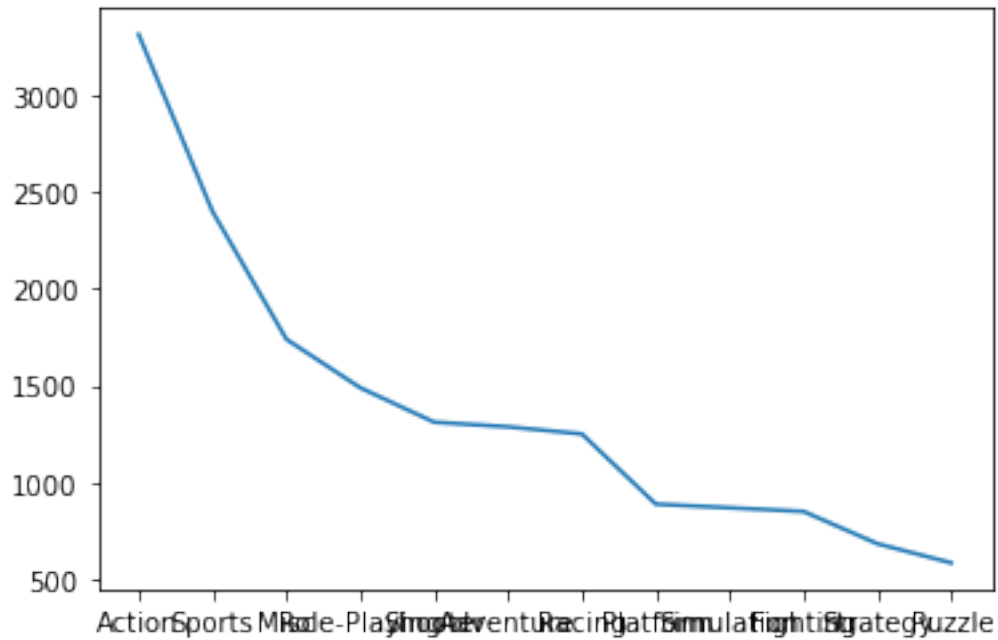
```
[59]: Puzzle            582  
Strategy            681  
Fighting            848  
Simulation           867  
Platform            886  
Racing             1249  
Adventure           1286  
Shooter            1310  
Role-Playing        1488  
Misc               1739  
Sports             2400  
Action             3316  
Name: Genre, dtype: int64
```

```
[31]: genre = data['Genre'].value_counts().index
```

```
[32]: value = data['Genre'].value_counts().values
```

```
[ ]:
```

```
[33]: plt.plot(genre, value)  
plt.show() # line chart
```

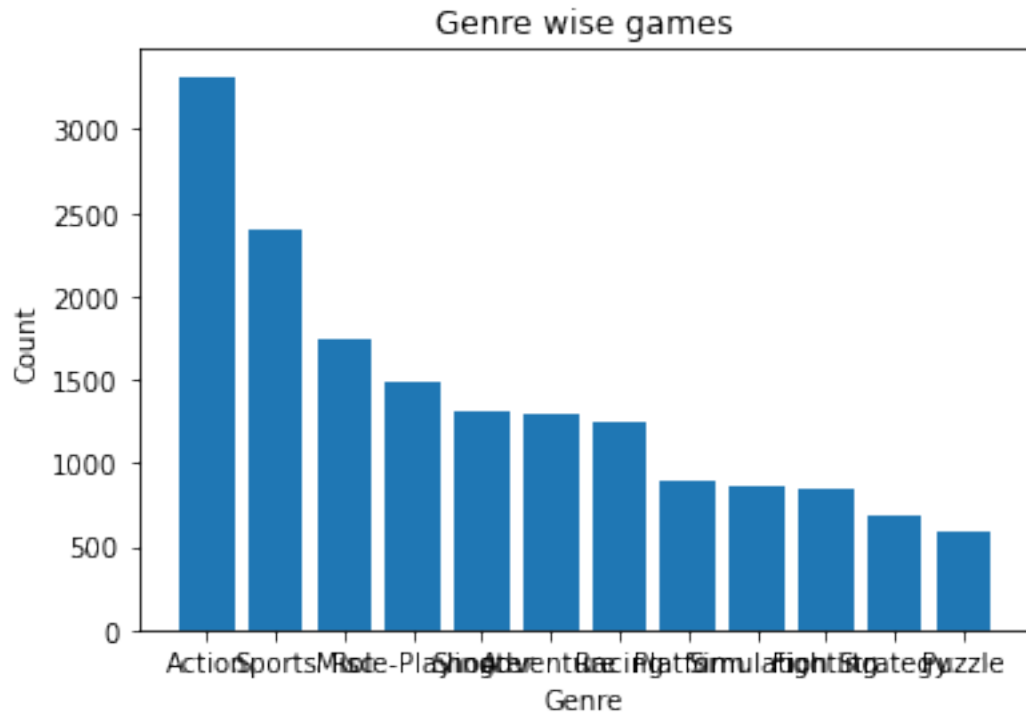


[]:

[48]: *# bar plot: When we want to see freq wrt to a value*

```
[34]: plt.bar(genre, value)

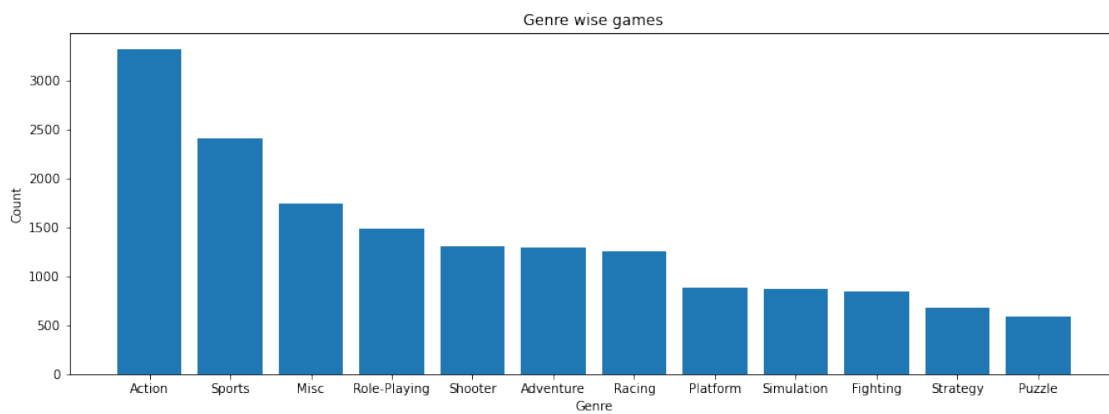
plt.title("Genre wise games")
plt.xlabel("Genre")
plt.ylabel("Count")
plt.show()
```



```
[28]: # Providing more customization here
```

```
[35]: plt.figure(figsize=(15, 5)) # (x, y)
```

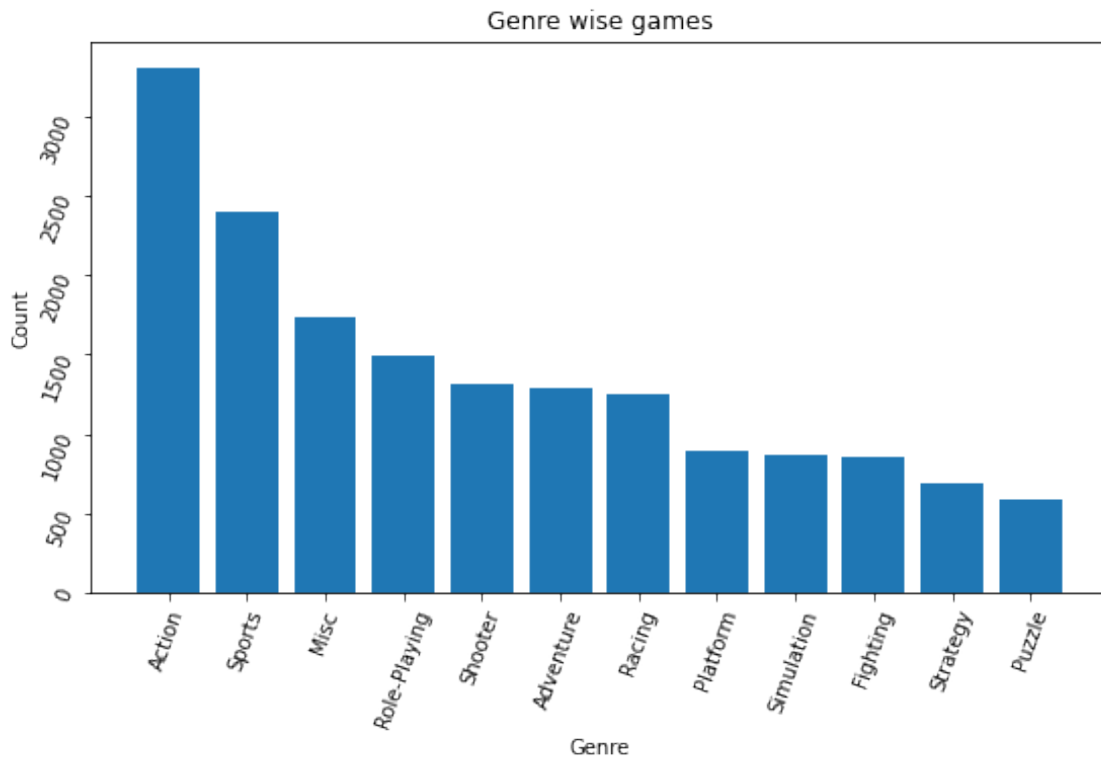
```
plt.bar(genre, value)
plt.title("Genre wise games")
plt.xlabel("Genre")
plt.ylabel("Count")
plt.show()
```



```
[39]: plt.figure(figsize=(9, 5)) # (x, y)
```

```
plt.bar(genre, value)
plt.title("Genre wise games")
plt.xlabel("Genre")
plt.ylabel("Count")
plt.xticks(rotation = 70)
plt.yticks(rotation = 70)

plt.show()
```



```
[50]: data.head()
```

```
[50]:
```

	Rank	Name	Platform	Year	Genre	\
0	2061	1942	NES	1985.0	Shooter	
1	9137	¡Shin Chan Flipa en colores!	DS	2007.0	Platform	
2	14279	.hack: Sekai no Mukou ni + Versus	PS3	2012.0	Action	
3	8359	.hack//G.U. Vol.1//Rebirth	PS2	2006.0	Role-Playing	
4	7109	.hack//G.U. Vol.2//Reminisce	PS2	2006.0	Role-Playing	

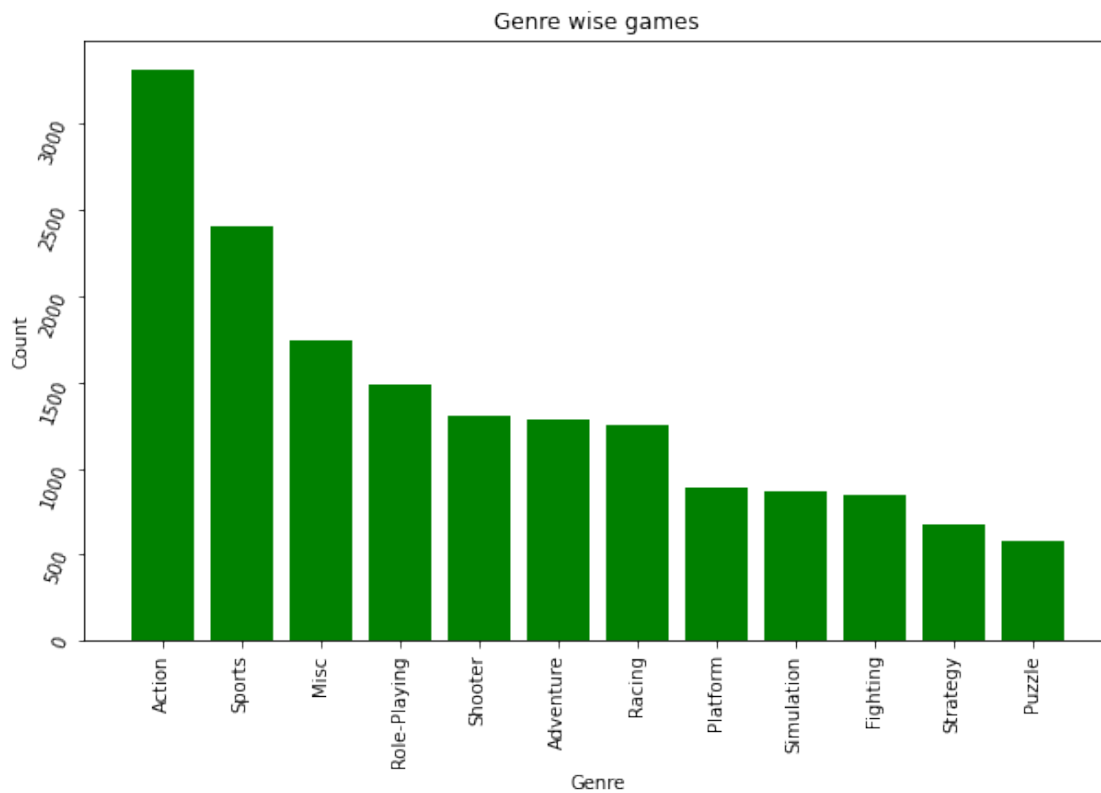
	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
0	Capcom	4.569217	3.033887	3.439352	1.991671	12.802935

1	505 Games	2.076955	1.493442	3.033887	0.394830	7.034163
2	Namco Bandai Games	1.145709	1.762339	1.493442	0.408693	4.982552
3	Namco Bandai Games	2.031986	1.389856	3.228043	0.394830	7.226880
4	Namco Bandai Games	2.792725	2.592054	1.440483	1.493442	8.363113

```
[47]: plt.figure(figsize=(10, 6)) # (x, y) -> size of graph
```

```
plt.bar(genre, value, color = 'green')
plt.title("Genre wise games")
plt.xlabel("Genre")
plt.ylabel("Count")
plt.xticks(rotation = 90)
plt.yticks(rotation = 70)

plt.show()
```

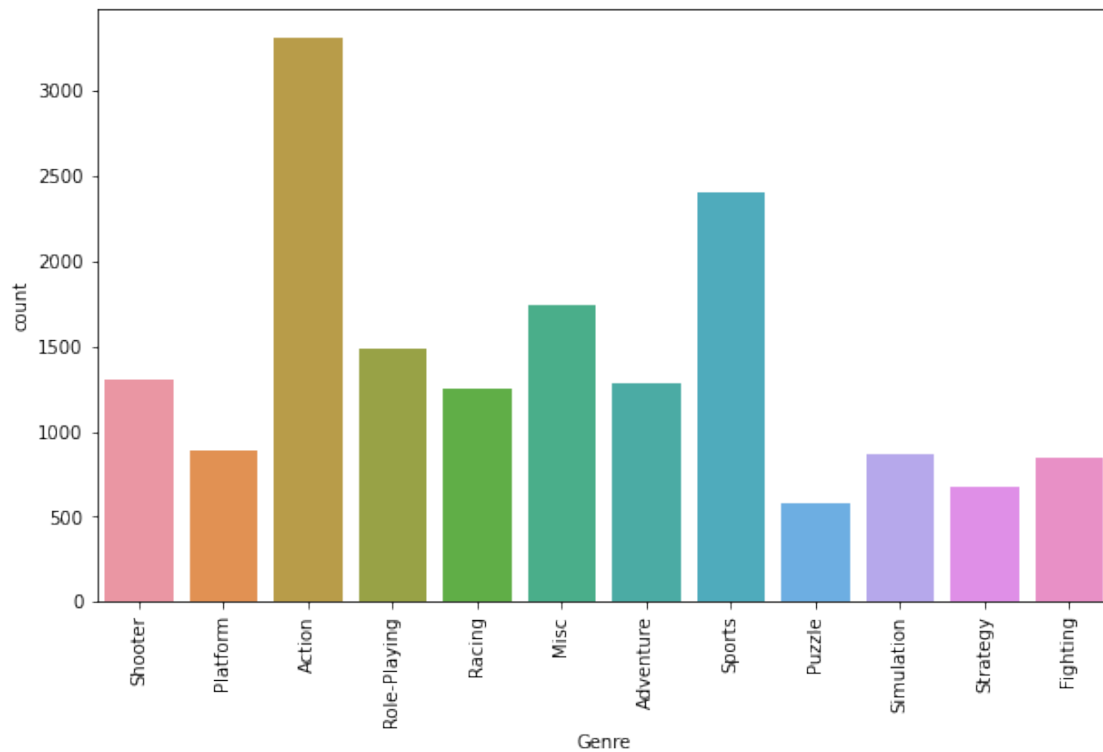


```
[49]: # seaborn
```

```
[55]: plt.figure(figsize=(10, 6))
sns.countplot(data = data, x = 'Genre') # barplot
plt.xticks(rotation = 90)
```



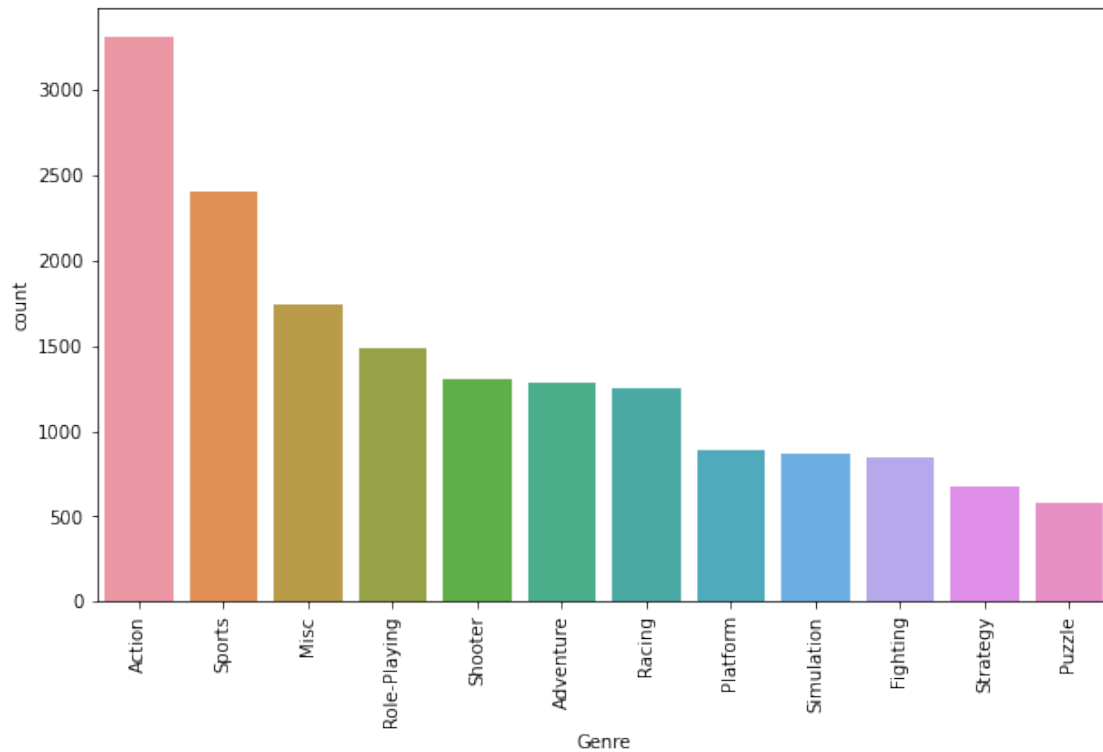
```
plt.show()
```



```
[ ]:
```

```
[56]: plt.figure(figsize=(10, 6))
sns.countplot(data = data, x = 'Genre', order = data['Genre'].value_counts().
↳index) # barplot
plt.xticks(rotation = 90)

plt.show()
```



[]:

[]:

0.0.4 Histograms

- Frequency with bins

[63]: `data.head()`

```
[63]:
```

	Rank	Name	Platform	Year	Genre	\
0	2061	1942	NES	1985.0	Shooter	
1	9137	Shin Chan Flipa en colores!	DS	2007.0	Platform	
2	14279	.hack: Sekai no Mukou ni + Versus	PS3	2012.0	Action	
3	8359	.hack//G.U. Vol.1//Rebirth	PS2	2006.0	Role-Playing	
4	7109	.hack//G.U. Vol.2//Reminisce	PS2	2006.0	Role-Playing	

	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
0	Capcom	4.569217	3.033887	3.439352	1.991671	12.802935
1	505 Games	2.076955	1.493442	3.033887	0.394830	7.034163
2	Namco Bandai Games	1.145709	1.762339	1.493442	0.408693	4.982552
3	Namco Bandai Games	2.031986	1.389856	3.228043	0.394830	7.226880
4	Namco Bandai Games	2.792725	2.592054	1.440483	1.493442	8.363113

```
[66]: data['Year'].nunique()
```

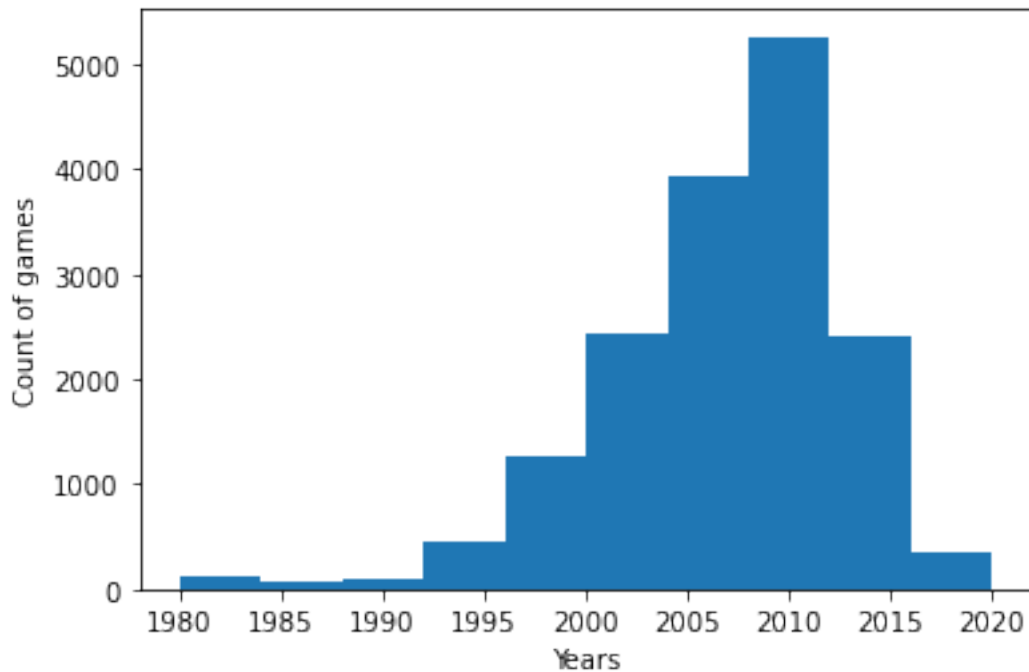
```
[66]: 41
```

```
[67]: data['Year'].min(), data['Year'].max()
```

```
[67]: (1980.0, 2020.0)
```

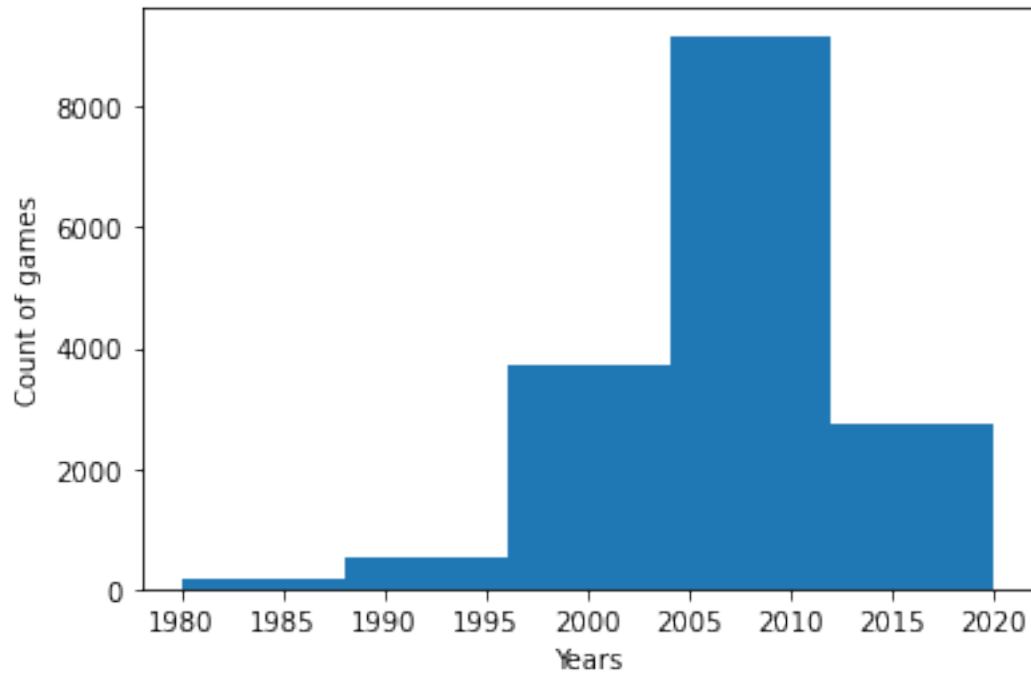
```
[70]: # By default histogram offers 10 bins to us
```

```
[73]: plt.hist(data['Year'])  
plt.xlabel("Years")  
plt.ylabel("Count of games")  
plt.show()
```



```
[77]: # This above data is left skew: That the density lies on right side.
```

```
[76]: plt.hist(data['Year'], bins = 5)  
plt.xlabel("Years")  
plt.ylabel("Count of games")  
plt.show()
```



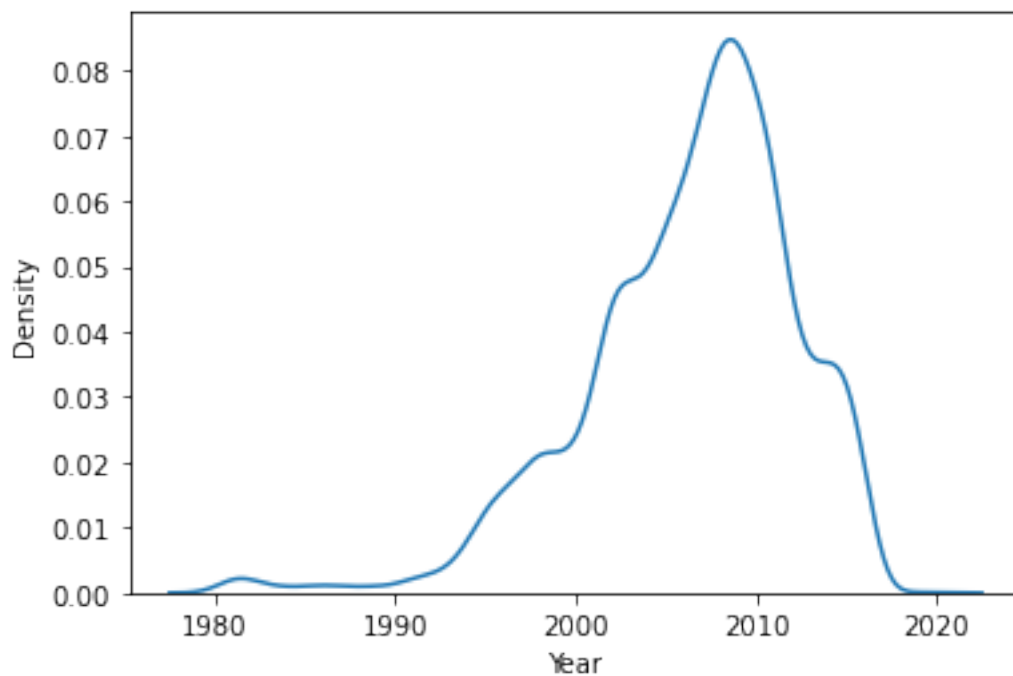
[]:

0.0.5 Kernel Density Estimate (KDE) plot

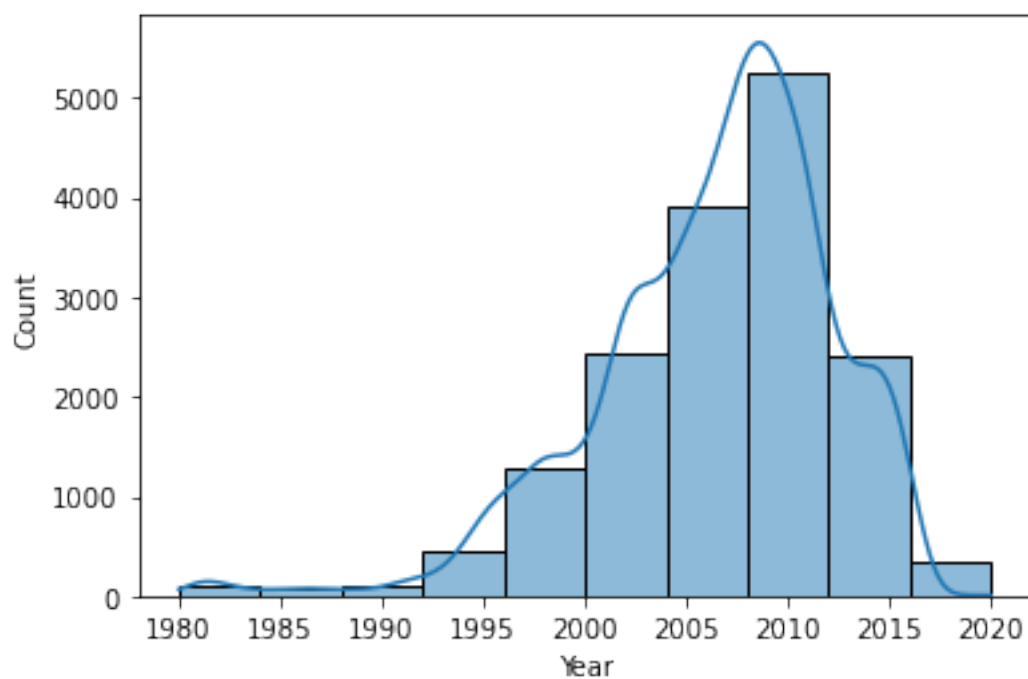
- Smoothened version of Histogram
- Used to visualize density composition of data

[]:

```
[81]: sns.kdeplot(data = data, x = 'Year')  
plt.show()
```



```
[82]: sns.histplot(data = data, x = 'Year', bins = 10, kde = True)  
plt.show()
```



```
[ ]:
```

0.0.6 Boxplot

- Also known as box and whiskers plot
 - Used to find 5 point summary of data
1. Minimum score
 2. First(lower) quartile (25 quartile)
 3. Second Quartile -> Median -> (50 quartile)
 4. Third Quartile -> (75 quartile)
 5. Maximum score

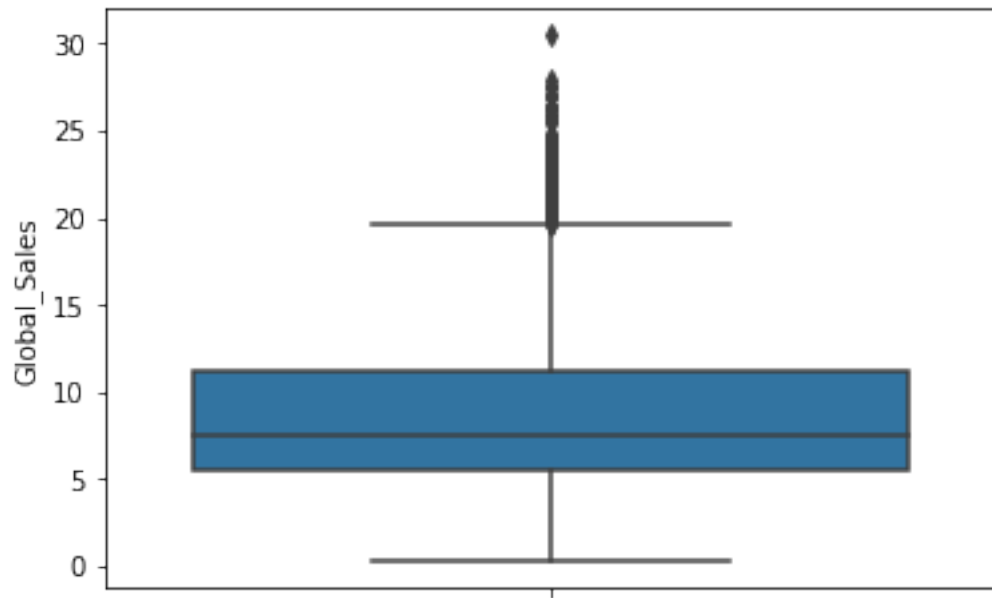
```
[83]: data.head()
```

```
[83]:
```

	Rank	Name	Platform	Year	Genre	\
0	2061	1942	NES	1985.0	Shooter	
1	9137	¡Shin Chan Flipa en colores!	DS	2007.0	Platform	
2	14279	.hack: Sekai no Mukou ni + Versus	PS3	2012.0	Action	
3	8359	.hack//G.U. Vol.1//Rebirth	PS2	2006.0	Role-Playing	
4	7109	.hack//G.U. Vol.2//Reminisce	PS2	2006.0	Role-Playing	

	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
0	Capcom	4.569217	3.033887	3.439352	1.991671	12.802935
1	505 Games	2.076955	1.493442	3.033887	0.394830	7.034163
2	Namco Bandai Games	1.145709	1.762339	1.493442	0.408693	4.982552
3	Namco Bandai Games	2.031986	1.389856	3.228043	0.394830	7.226880
4	Namco Bandai Games	2.792725	2.592054	1.440483	1.493442	8.363113

```
[84]: sns.boxplot(data = data, y = 'Global_Sales')
plt.show()
```



```
[88]: data['Global_Sales'].median()
```

```
[88]: 7.536613645
```

```
[87]: data['Global_Sales'].max()
```

```
[87]: 30.55586186
```

```
[86]: data['Global_Sales'].min()
```

```
[86]: 0.24
```

```
[90]: # Quiz
```

- 1. For analyzing marks given by an edtech company, we want to find the range in which the most number of students have scored. Which will be the best suited plot for this? Ans -> Histogram
- 2. The telecom company “Airtel” wants to find the count customers across different payment modes opted by the customer. Which will be the best suited plot for this? Ans: countplot/barplot
- 3. In the state “Haryana”, we want to find the proportion of people who smoke. Which will be the best suited plot for this? Ans: Piechart

```
[ ]:
```

```
[ ]:
```

[]:

[]:

[]:

[]:

[]: